

Weekly Rapport — Weeks 10–11

Real-Time Anomaly Detection Platform for Branch Teller Operations

Amen Bank — Direction Centrale du Systeme d'Information
Departement des Projets Agence

July 8–16, 2026

Sessions covered	2 (July 14, July 15–16)
Primary deliverables	Sink consumers, schema redesign, batch enrichment rewrite, Airflow orchestration
Services added	raw-events-sink, decisions-sink, airflow-init, airflow-webserver, airflow-scheduler
Docker Compose total	17 services (up from 12)
DAGs validated E2E	nightly_profiles (8 tasks), weekly_retrain (3 tasks)

1 Executive Summary

This rapport covers two sessions (July 14 and July 15–16), during which the project advanced on two major fronts: closing the gap between streaming output and persistent storage, and building the complete batch orchestration layer with Apache Airflow.

Session 1 began with the intent to build the Streamlit supervisor dashboard but pivoted when a fundamental architectural gap was identified: the streaming pipeline produced decisions on a Kafka topic, but no component persisted those decisions to PostgreSQL or Redis for the dashboard to read. This led to the design and implementation of two sink consumers, a schema redesign for the alerts and evaluation_labels tables, and a thorough debugging process that ultimately uncovered a critical feature alignment issue between batch training and streaming inference.

Session 2 resolved that feature alignment issue by rewriting batch_enrichment.py to reuse the streaming EnrichmentCore's pure feature computation function, then wired Apache Airflow with two DAGs — a nightly profile rebuild with validation gates and a weekly retraining pipeline. Both DAGs were validated end-to-end inside Docker, including resolution of memory constraints, environment variable mismatches, and shared log configuration.

2 Sink Consumers and Schema Redesign

2.1 Architectural Gap: Streamlit Cannot Consume Kafka

The Streamlit dashboard requires data from PostgreSQL and Redis, not directly from Kafka. Streamlit's script re-run model (the entire script re-executes on every user interaction) is incompatible with Kafka's consumer group protocol, which requires long-lived consumer instances maintaining partition assignments and offsets. This necessitates a decoupled sink pattern: dedicated consumer services write to PostgreSQL and Redis, and the dashboard reads from those stores.

2.2 Raw Events Sink

The raw-events sink consumes from the raw-events topic and writes each transaction to the transactions table in PostgreSQL. This provides a persistent record of every event that enters the pipeline, enabling batch processing, auditing, and dashboard queries.

2.3 Decisions Sink

The decisions sink consumes from the decisions topic and performs two operations:

- **PostgreSQL:** Inserts alert records into the redesigned alerts table with tier, fused score, reasons, regulatory flags, SHAP explanations, and supervisor decision fields
- **Redis:** Updates risk history counters (alert counts per client) — scoped to risk history only, since the enrichment service already handles behavioral buffer updates

2.4 Schema Redesign

Three schema changes were made to support the sink architecture:

- **transactions table:** Oracle label columns (is_anomaly, anomaly_type) removed — these are ground-truth labels that should never exist in a production transactions table

- **evaluation_labels table:** Created with a safe FK to transactions (batch-only context, no race condition risk). Stores is_anomaly and anomaly_type for model evaluation
- **alerts table:** Rebuilt with branch, amount, operation type, regulatory flags, SHAP explanation (JSONB), and supervisor decision fields. **The FK from alerts to transactions was deliberately removed** to avoid a race condition between the two sink consumers, which consume from different topics in parallel. Referential integrity is guaranteed by pipeline topology — a decision event cannot exist without a corresponding raw event having entered the pipeline.

3 Feature Alignment Discovery

End-to-end pipeline testing revealed that anomaly scores were systematically elevated across the board. Investigation traced the root cause to a feature distribution mismatch:

- **Training:** batch_enrichment.py used the old 74-feature schema with oracle shortcuts (access to ground-truth labels and full historical context at enrichment time)
- **Inference:** The streaming EnrichmentCore computes 30 contrast features using real-time profile data and rolling buffers

The models were trained on a fundamentally different feature distribution than what they receive at inference time. This manifested as inflated scores because the model's learned normal distribution did not match the streaming feature distribution. Resolution of this issue became the top priority, ahead of the dashboard.

4 Debugging and Infrastructure Fixes

Session 1 included an extensive debugging process that uncovered several issues:

Issue	Root Cause	Fix
Docker stdout buffering	Python output buffered in Docker containers	PYTHONUNBUFFERED=1 in Dockerfile.api
Wrong PostgreSQL instance	Local PostgreSQL on port 5432 conflicting with Docker	Batch scripts use DB_CONFIG with env vars
Empty profile_snapshots	Batch jobs not in Docker Compose startup	Documented as Airflow gap; manual re-run required
Redis connection default	hydrate_redis.py defaulting to localhost	Read REDIS_HOST/PORT from src.config
Stale PostgreSQL connection	Consumer group offset reset causing silent INSERT failures	Connection refresh in decisions sink
Buffer contamination	Multiple simulator runs accumulating Redis buffer state	Clean test procedure: FLUSHDB, re-hydrate, single run

5 Batch Enrichment Rewrite

5.1 EnrichmentCore Purification

Before the batch enrichment could reuse the streaming feature logic, one remaining Redis call inside `_compute_features` had to be removed. The archetype baseline fallback (triggered when a client has zero transactions of a given operation type in both 30-day and 90-day windows) previously called `self.redis.get(f"baseline:{{arch}}")` directly. This was refactored: the baseline is now passed as a parameter (`archetype_baseline=None`), fetched by the caller (either `enrich_event` for streaming, or the batch script for batch). The function now has zero Redis calls — pure inputs in, features out.

5.2 Batch Script Design

The rewritten `batch_enrichment.py` follows this architecture:

- **Single source of truth:** Calls `EnrichmentCore._compute_features()` directly — same function, same 30 features, zero drift risk between batch and streaming
- **Server-side cursor:** PostgreSQL streams transactions in chunks of 5,000 via a named cursor, avoiding the OOM kill that occurred when loading all 243k rows into memory
- **In-memory buffers:** Client and employee buffers are maintained as Python dictionaries during chronological iteration, mimicking the Redis buffer behavior from streaming
- **Profile snapshot matching:** For each transaction at time T , the script finds the latest profile snapshot where `computed_at < T`, ensuring temporal consistency
- **Incremental CSV output:** Results are written per chunk via `csv.DictWriter`, keeping memory footprint constant regardless of dataset size

5.3 Memory Optimization

Initial runs inside Docker were OOM-killed even with chunked transaction loading. The remaining memory pressure came from profile snapshots: 10,000 clients x 26 weekly snapshots = 260,000 profile blobs, each containing a `recent_events_buffer` of 50 events that `_compute_features` never reads. Stripping this field at load time (`data.pop("recent_events_buffer", None)`) cut memory usage enough for the job to complete successfully in Docker.

5.4 Output Validation

The enriched training dataset was validated:

Metric	Value
Total rows	243,904
Feature columns	30 (from <code>feature_cols.json</code>)
Output columns	34 (30 features + <code>event_id</code> , <code>client_id</code> , <code>is_anomaly</code> , <code>anomaly_type</code>)
<code>z_amount</code> distribution	Mean 4.70, Std 489.57, range [-15,770 to 28,660]
<code>tx_count_24h</code> range	0 to 13
<code>tx_count_7d</code> range	0 to 38
Zero-variance columns	1 (<code>day_distance_from_preferred</code> — hardcoded placeholder)
Evaluation labels	243,904 matched (100%)

6 Idempotency Fixes for Airflow Retries

All three batch scripts were updated with ON CONFLICT upsert clauses to handle Airflow's automatic retry behavior. Without these, a retry of a partially completed batch job would crash on duplicate primary key violations.

Script	Table	Conflict Key	Action
batch_profile.py	profile_snapshots	(client_id, computed_at)	UPDATE profile_data
batch_employee_profile.py	employee_profile_snapshots	(employee_id, computed_at)	UPDATE profile_data
batch_archetype_baselines.py	archetype_baselines	(archetype)	UPDATE baseline_data

7 Apache Airflow Integration

7.1 Architecture Decisions

Airflow was integrated with the following design choices:

- **LocalExecutor:** Runs tasks in the scheduler process — sufficient for branch teller volumes. Switch to CeleryExecutor is config-only (same DAGs, add worker containers)
- **Shared PostgreSQL:** Airflow metadata tables coexist with application tables in the same database to conserve memory on a 16GB development machine at ~74% utilization
- **Lightweight Airflow image:** Custom Dockerfile.airflow installs only requirements-airflow.txt (redis, numpy, pandas, psycpg2-binary, python-dotenv) — no PyTorch, FastAPI, or SHAP, saving ~2GB of image size
- **Volume-mounted code:** Project directory mounted at /app with PYTHONPATH=/app, so code changes take effect on restart without image rebuilds

7.2 Nightly Profiles DAG

The nightly_profiles DAG runs at 2:00 AM daily and rebuilds all behavioral profiles. Each batch task is followed by a validation gate that stops the pipeline if output is missing or corrupt — preventing the downstream propagation of bad data that caused extensive debugging in earlier sessions.

Task	Type	Validation Checks
run_batch_profile	BashOperator	—
validate_profiles	PythonOperator	snapshot_count > 0; client coverage >= 95%; no NULL profile_data
run_batch_employee_profile	BashOperator	—
validate_employee_profiles	PythonOperator	employee coverage >= 95%; no NULL profile_data
run_batch_archetype_baselines	BashOperator	—

Task	Type	Validation Checks
validate_baselines	PythonOperator	exactly 5 baselines; no NULL baseline_data
run_hydrate_redis	BashOperator	—
validate_redis	PythonOperator	profile:client:* > 0; buffer:client:* > 0; baseline:* == 5; profile:employee:* > 0

7.3 Weekly Retrain DAG

The weekly_retrain DAG runs at 4:00 AM every Sunday, after the nightly DAG has refreshed profiles. It produces the training dataset and validates feature completeness.

Task	Type	Details
run_batch_enrichment	BashOperator	Produces data/training_features.csv
validate_enrichment	PythonOperator	Row count > 0; all 30 feature columns present; NaN rate < 5% per column
retrain_models	BashOperator	Placeholder — to be wired with MLflow

7.4 E2E Validation Results

Both DAGs were triggered manually in Docker and completed successfully:

- **nightly_profiles:** 8/8 tasks green. run_batch_profile completed in ~41 minutes (26 weekly snapshots x 10,000 clients). All validation gates passed.
- **weekly_retrain:** 3/3 tasks green. batch_enrichment processed 243,904 events with chunked streaming. validate_enrichment confirmed 34 output columns and 243,904 matched evaluation labels.

8 Docker Compose Evolution

The Docker Compose stack grew from 12 to 17 services with the addition of sink consumers and Airflow infrastructure.

Service	Port	Role
redpanda	9092	Kafka-compatible message broker
redis	6379	Hot profiles, buffers, baselines
postgres	5432	Cold storage + Airflow metadata
simulation-api	8000	FastAPI synchronous scoring
enrichment-service	9100	Streaming enrichment + /metrics
scoring-service	9101	Streaming scoring + /metrics
decision-service	9102	Streaming decisions + /metrics
raw-events-sink	9103	raw-events topic to PostgreSQL
decisions-sink	9104	decisions topic to PostgreSQL + Redis

Service	Port	Role
kafka-exporter	9308	Consumer lag metrics
prometheus	9090	Metrics aggregation
hydrate-redis	—	Profile hydration (startup)
db-seed	—	Initial data load (on-demand)
simulator	—	Event replay (on-demand)
airflow-init	—	DB migration + admin user (one-shot)
airflow-webserver	8080	DAG visualization and management
airflow-scheduler	—	Task execution and scheduling

9 Issues Resolved During Airflow Integration

Issue	Symptom	Resolution
Environment variable naming	batch_profile.py connecting to localhost:5432 instead of postgres:5432	Airflow anchor was setting POSTGRES_HOST but src.config reads DB_HOST — corrected to match
Log sharing between containers	403 Forbidden when reading task logs from webserver	Added shared AIRFLOW__WEBSERVER__SECRET_KEY and airflow-logs volume
OOM kill (attempt 1)	batch_enrichment killed with exit code -9 loading 243k rows	Server-side cursor with CHUNK_SIZE=5000
OOM kill (attempt 2)	Still killed after chunking transactions	Stripped recent_events_buffer from profile snapshots at load time
Database name mismatch	Airflow anchor defaulting to amen_bank instead of amen_anomaly	Hardcoded SQL_ALCHEMY_CONN string and aligned DB_NAME defaults

10 Files Modified / Created

File	Action
config/init.sql	Rebuilt (alerts, evaluation_labels, dropped FKs)
scripts/load_postgresql.py	Updated (split labels to evaluation_labels)
scripts/debug_single_event.py	Created (pipeline comparison tool)
src/enrichment/core.py	Modified (archetype_baseline parameter, pure_compute_features)
src/decision/core.py	Modified (enriched_features in output)
src/streaming/sinks/__init__.py	Created

File	Action
src/streaming/sinks/raw_events_sink.py	Created
src/streaming/sinks/decisions_sink.py	Created
src/batch/batch_enrichment.py	Rewritten (chunked, reuses EnrichmentCore)
src/batch/batch_profile.py	Modified (ON CONFLICT upsert)
src/batch/batch_employee_profile.py	Modified (ON CONFLICT upsert)
src/batch/batch_archetype_baselines.py	Modified (ON CONFLICT upsert)
src/batch/hydrate_redis.py	Modified (REDIS_HOST/PORT from config)
src/streaming/simulator.py	Modified (burst_mode, offset param)
Dockerfile.api	Modified (PYTHONUNBUFFERED=1)
Dockerfile.airflow	Created (lightweight Airflow image)
requirements-airflow.txt	Created (minimal deps for Airflow)
dags/nightly_profiles.py	Created (8-task DAG with validation gates)
dags/weekly_retrain.py	Created (3-task retraining DAG)
docker-compose.yml	Rebuilt (17 services, Airflow anchor, shared logs)
config/prometheus.yml	Modified (sink scrape targets)

11 Next Steps

- **Streamlit dashboard:** Now unblocked — consumes alerts from PostgreSQL, pipeline health from Prometheus, displays SHAP explanations, KPIs per branch/operation, simulation mode, supervisor feedback
- **MLflow integration:** Model versioning and experiment tracking, wired into the retrain_models task placeholder
- **Model retraining:** Retrain AE and LSTM on the new 30-feature training dataset produced by batch_enrichment — resolves the feature alignment gap
- **Feedback loop:** Supervisor decisions from the dashboard feed back into the retraining pipeline via evaluation_labels
- **Structured JSON logging:** Replace print statements with structured logging across all services for centralized log aggregation